

APEX-SQL

通过智能体探索，与数据对话

Talking to the Data via Agentic Exploration for Text-to-SQL

Yushi Sun | 光子技术发展部 | 2026.08

从“生成 SQL”走向“验证假设”



个人简介

研究兴趣与本次工作定位



Yushi Sun
孫宇石

光子技术发展部



研究方向

Data Agent
Game Agent



关注问题

语义理解
数据 Grounding
复杂任务推理
智能体记忆



本次工作目标

让自然语言
更准确、更可靠地
访问企业数据

论文信息

APEX-SQL: Talking to the Data via Agentic Exploration for Text-to-SQL

KDD 2026

APEX-SQL

Talking to the Data via Agentic Exploration
for Text-to-SQL

Bowen Cao · Weibin Liao · Yushi Sun · Dong Fang · Haitao Li · Wai Lam

arXiv:2602.16720v2 | cs.DB / cs.AI | Code: github.com/Tencent/APEX-SQL-Project



论文一句话

让 Text-to-SQL Agent
主动探索真实数据，
通过“假设—验证”
提升企业级语义准确性

学术 Benchmark 很强，企业数据库仍然很难

问题不只在 SQL 语法，而在数据语义与规模

01



静态 Schema 失明

列名模糊、缩写不透明、领域编码隐藏在数据值中。模型“看见结构”，却看不见真实含义。

02



大规模搜索空间

企业数据库可包含数百张表、数千列。直接塞入上下文带来噪声与高昂 token 开销。

03



语法正确 ≠ 语义正确

SQL 可以成功执行，但过滤条件、Join 路径或聚合口径错误，结果依然不可用。

核心判断：企业级 Text-to-SQL 必须从“被动翻译”升级为“主动验证”

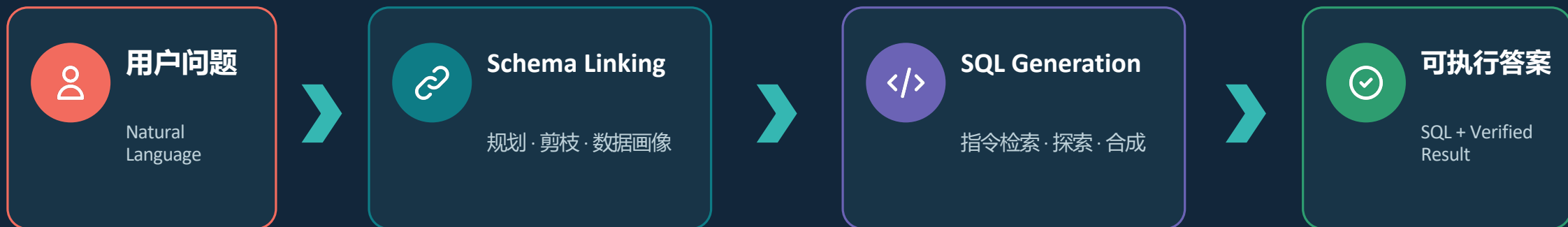
从被动感知到智能体探索

三种 Text-to-SQL 范式对比



APEX-SQL: 两阶段 Hypothesis–Verification Loop

Schema Linking 先找“足够且连通”的子图，SQL Generation 再在真实数据上探索与验证



Hypothesis

提出可检验的语义假设

Verification

查询真实数据并修正推理

同一个 H-V Loop 在两个阶段复用

Schema Linking: 从数千列到充分子图

目标: 高召回地找到相关列, 同时保持 Join 拓扑连通



设计取舍: 企业级场景优先保证关键列召回率, 宁可多保留一些候选列, 也不能漏掉唯一的正确路径

SQL Generation: Guidance 驱动的探索闭环

确定性指令检索 + 四类 Agent 动作

Deterministic Guidance

从逻辑计划中抽取 SQL 操作关键词，映射到 49 条 tips / 14 类规则。

RANK

检查排序列 NULL

DIVIDE

防零 + 浮点转换

STRING

验证大小写与模糊匹配

pilot 关键词召回率 > 95%

01 Profiling

执行探索 SQL，观察分布与样例

02 Consolidation

压缩状态，对齐计划与证据

03 SQL Synthesis

合成候选 SQL 并立即执行

04 Confirmation

终态复核语义，防止逻辑漂移

H-V 实例①: Schema Linking 怎么假设、怎么验证

示例问题: 哪些高中 (EILCode = HS) 的平均分最高?

HYPOTHESIS · 提出假设

① Logical Plan (不含具体表列名 · 防 schema bias)

Step 1: Filter for high schools

- Possible paths: school_type / EILCode / join school_types table
- Keywords: schools, EILCode, filter
- Evidence: EILCode = 'HS' 表示高中

② 角色假设 (Semantic Linking)

"table_1": "bridge table: 经 student_id 和 class_id 连接 Students 和 Classes"

VERIFICATION · 数据画像验证

```
-- Motivation: 检查 status 取值
-- 是否匹配过滤需求
SELECT DISTINCT status
FROM table_name LIMIT 10;
```

```
{
  "relevant": true,
  "relevance_reason": "提供 X↔Y 的 Join Key",
  "observations": "列 A 为整数编码 1-5"
}
```

证据说话: relevant = true 才保留; 拒绝列必须给出 reject_reason; Join 不通就再发 exploration_queries 补验

H-V 实例②：真实 Case (Spider 2.0-Snow)

问题：只看 NPM 包的最新 release 版本，按 GitHub star 数取 Top 8 的包及版本

BASELINE · 直接假设

不验证，直接假设
UpstreamPublishedAt 字段可用

```
WHERE  
  "UpstreamPublishedAt"  
  IS NOT NULL
```

43.7%

的 release 该字段为
NULL，被误排除

结果：Score 0 X

R1

检验假设：时间戳字段可用吗？

抽样 10 行 → UpstreamPublishedAt 全部为 NULL → 假设证伪

R2

量化影响：NULL 到底有多少？

COUNT → 393 万条 NULL，占约 900 万 release 的 43.7%，baseline 会漏掉近一半的包

R3

验证替代方案：确认可用后再写 SQL

VersionInfo:Ordinal 可排序；Join 键格式与 PROJECTS.Name 匹配（如 steven-tey/dub）

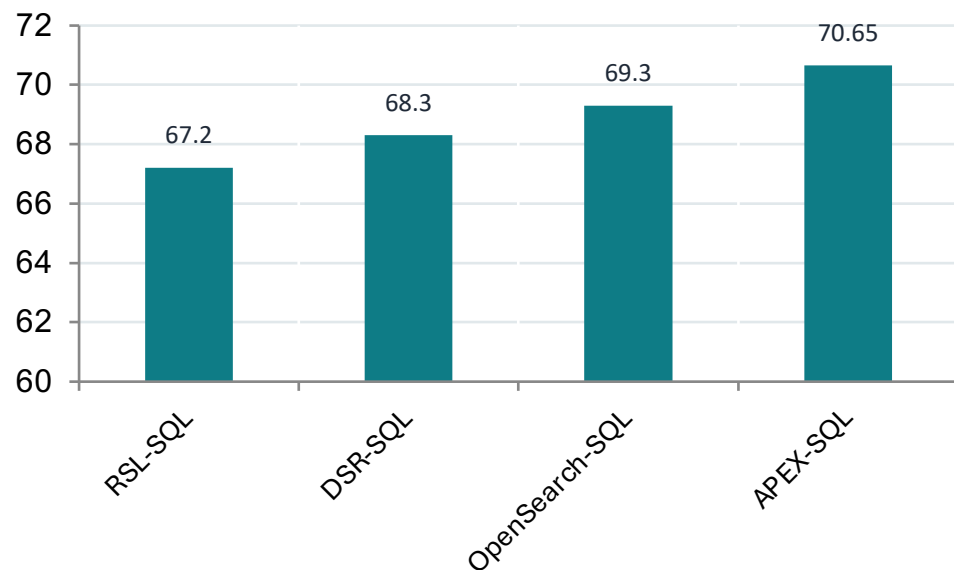
```
最终 SQL: ROW_NUMBER() OVER (PARTITION BY "Name"  
  ORDER BY "VersionInfo":Ordinal DESC) → Score: 1 ✓
```

Exploration is not guessing — it is systematic data quality verification.

关键结果：在两个企业级 Benchmark 上领先

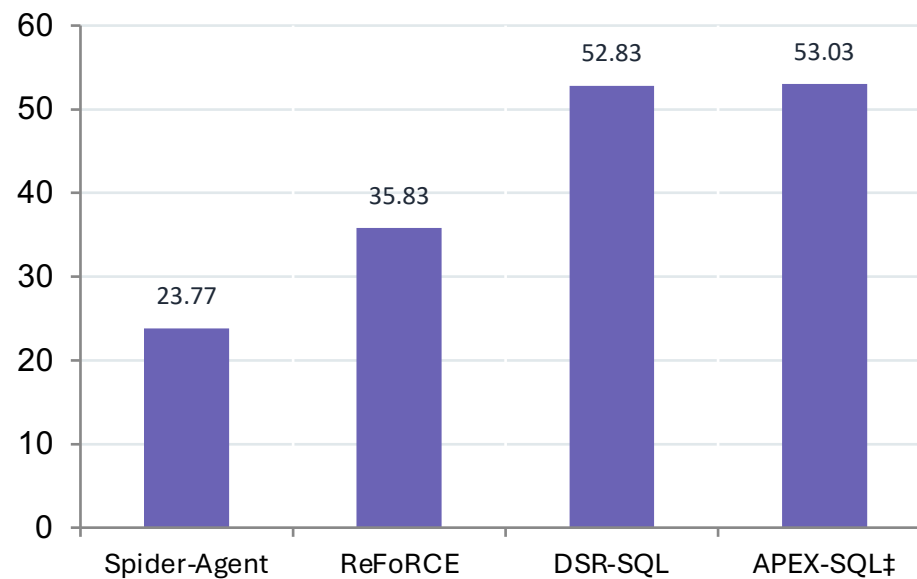
Execution Accuracy (%)

BIRD-Dev · Execution Accuracy



Spider 2.0-Snow · EX

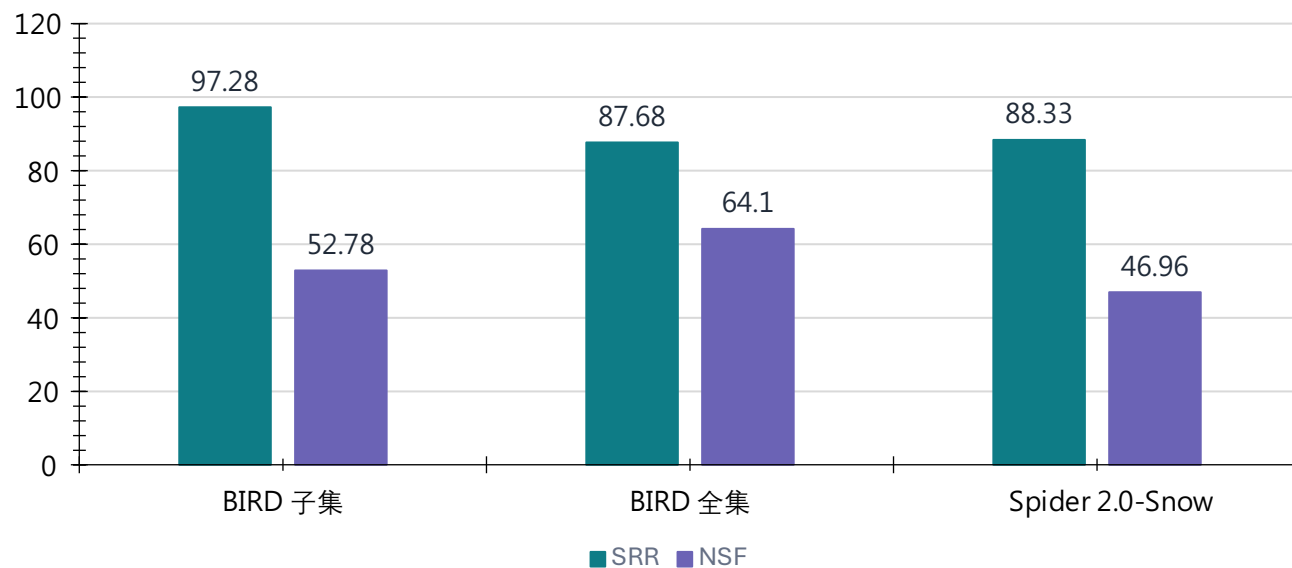
Pass@8 68.44%



Schema Linking: 高召回地定位关键数据

Table 1 : Strict Recall Rate (SRR) 与 Non-strict F1 (NSF) · 单位 : %

APEX-SQL Schema Linking · Table 1



Backbone : BIRD 子集 DeepSeek-V3.2 ; BIRD 全集 Qwen3-32B ; Spider 2.0-Snow GPT-4.1

弱模型场景仍保持稳健

87.68
%

BIRD 全集 SRR
AutoLink 39.50%

复杂企业库仍保持高召回

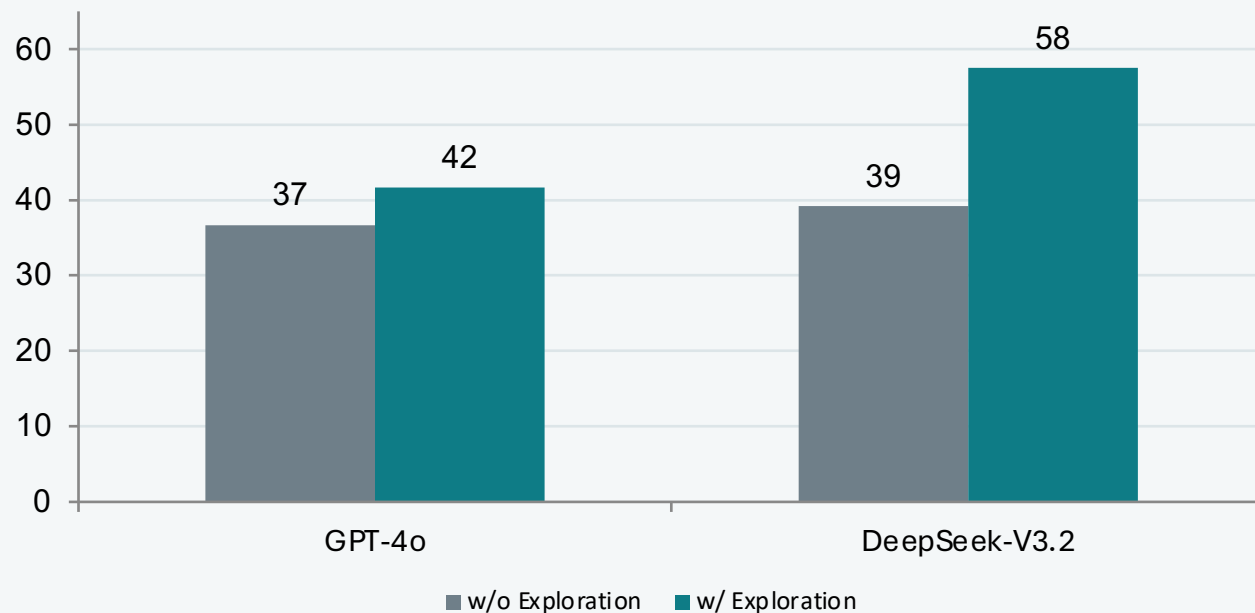
88.33
%

Spider 2.0 SRR
ReFoRCE 35.00%

SRR 优先 : 漏掉一个关键列就无法生成正确 SQL

Agentic Exploration 是“性能倍增器”

Oracle schema 子集 (N=120)：越强的模型，探索增益越明显



+18.33

DeepSeek-V3.2 EX 提升 (百分点)

Rich Get Richer

探索不是替代模型推理，
而是释放基础模型的潜在能力。

哪些组件最关键？

Schema Linking 与 SQL Generation 的消融结果

Schema Linking: 去掉组件后的 SRR 下降



Guidance 的价值

+3.75

GPT-4o EX@8 提升 (百分点)

同时减少探索轮数:
3.96 → 3.52

结论: 语义链接负责“想对”,
数据画像负责“证实”。

应用价值：从“能用”走向“可信”

面向企业数据分析与智能问数的落地启示



更准确

主动验证字段语义、值分布与 Join 路径，降低“执行成功但答案错误”。



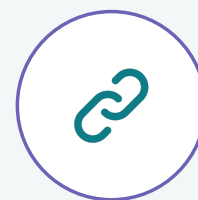
更可扩展

按需剪枝与并行画像，应对数百表、数千列的大规模数据库。



更高效

确定性 tips 减少无意义探索，以更少 token 获得更高准确率。



更可解释

保留“假设—证据—SQL”链路，便于调试、审计与人工复核。

典型场景：智能问数 · 数据分析 Copilot · BI 自助分析 · 企业知识与数据库联合查询

三个结论

01 企业 Text-to-SQL 的本质

不是翻译 schema，而是理解并验证真实数据语义。

02 APEX-SQL 的核心机制

用 Hypothesis-Verification Loop 贯穿 Schema Linking 与 SQL Generation。

03 最重要的实验洞见

Agentic Exploration 是性能倍增器，尤其能释放强模型潜力。
。

Q & A

谢谢

Paper: [arXiv:2602.16720](https://arxiv.org/abs/2602.16720)
Code: github.com/Tencent/APEX-SQL-Project